

Plano de Segurança — feat/security

Documento de referência para todos os IDs de segurança do branch `feat/security`.
Cada entrada inclui: o que é, porquê importa, como foi implementado e como verificar.

AUTH-07 — Rate Limiting nos Endpoints de Autenticação (08/05/2026)

O que é

Limite de pedidos por IP num dado período de tempo nos endpoints de autenticação.

Porquê importa

Sem rate limiting, um atacante pode fazer chamadas ilimitadas a `/auth/login` para adivinhar palavras-passe (brute force), registar milhares de contas falsas, ou abusar do envio de e-mails de reset.

Implementação

- **Biblioteca:** `flask-limiter[redis]>=3.5`
- **Backend de contadores:** Upstash Redis EU (Frankfurt) — partilhado entre os 4 workers gunicorn
- **Fallback dev:** `memory://` quando `UPSTASH_REDIS_URL` não está definida
- **Env var:** `UPSTASH_REDIS_URL` (formato: `redis://default:<token>@<host>:<port>`)

Endpoint	Limite
<code>POST /auth/login</code>	5 por minuto por IP
<code>POST /auth/register</code>	3 por hora por IP
<code>POST /auth/forgot-password</code>	3 por hora por IP
<code>POST /auth/reset-password</code>	5 por hora por IP

- `@app.errorhandler(429)` retorna `{"error": "RATE_LIMITED", "message": "..."} em PT-PT`
- `postJson` em `api.ts` trata 429 com mensagem PT-PT específica

Como verificar

```
# 6 pedidos seguidos ao /auth/login – o 6º deve retornar 429
for i in 1..6; do curl -s -X POST .../auth/login -H "Content-Type: application/json" -d
'{"username":"x","password":"y"}'; done
```

Confirmar contadores no Data Browser Upstash (chave `LIMITS:LIMITER/<ip>/auth_login/5/1/minute`).

AUTH-08 — Regenerar Sessão Após Login (08/05/2026)

O que é

Limpar a sessão Flask existente e criar uma nova imediatamente após autenticação bem-sucedida.

Porquê importa

Session fixation: um atacante pode tentar forçar a vítima a usar um session token que controla, e depois usá-lo após o login. Limpar a sessão antes de definir os dados de autenticação elimina qualquer estado residual e invalida qualquer token pré-login.

Em Flask com cookies assinados, o risco clássico de session fixation é baixo (o cookie é re-assinado com o `secret_key`), mas limpar a sessão é boa prática de compliance (OWASP ASVS V3.3).

Implementação

Adicionado `session.clear()` antes de `session["chichorro_auth"] = 1` nos 3 pontos de login em `Flask.py`:

1. Login por utilizadores hardcoded (env vars `CHICHORRO_AUTH_USER_N`)
2. Login em modo debug (sem env vars configuradas)
3. Login por utilizadores da base de dados (`users` table)

Como verificar

Login → DevTools → Application → Cookies: o valor do cookie de sessão (`session`) deve ter um valor diferente antes e após o login.

AUTH-06 — Hardening de Cookies de Sessão (08/05/2026)

O que é

Verificação e configuração explícita dos atributos de segurança do cookie de sessão Flask.

Porquê importa

- `HttpOnly`: impede JavaScript de aceder ao cookie (proteção XSS)
- `Secure`: garante que o cookie só é enviado em HTTPS
- `SameSite=Lax`: proteção parcial contra CSRF
- Nome `chichorro_session`: evita fingerprinting automático do Flask (o nome padrão `session` é um identificador óbvio)

Implementação

Configurações em `Flask.py`:

```
app.config["SESSION_COOKIE_HTTPONLY"] = True # hardcoded ✓
app.config["SESSION_COOKIE_SECURE"] = _session_secure # via
CHICHORRO_SESSION_SECURE=1 ✓
app.config["SESSION_COOKIE_SAMESITE"] = _session_same_site # via
CHICHORRO_SESSION_SAMESITE ✓
app.config["SESSION_COOKIE_NAME"] = "chichorro_session" # anti-fingerprinting ✓
```

`CHICHORRO_SESSION_SECURE=1` confirmada no Render produção.

Como verificar

DevTools → Application → Cookies após login:

- Cookie deve chamar-se `chichorro_session`
- Flags: HttpOnly ✓, Secure ✓ (produção), SameSite=Lax ✓

SEC-01 ✓ — Rever Configuração CORS (12/05/2026)

O que é

Revisão da configuração de Cross-Origin Resource Sharing para garantir que só as origens autorizadas acessem à API.

Porquê importa

CORS demasiado permissivo pode expor a API a ataques de outros domínios, incluindo CSRF via pedidos cross-origin com credenciais.

Implementação

```
_cors_origins = os.environ.get("CHICHORRO_CORS_ORIGINS")
if _cors_origins:
    _origins_list = [o.strip() for o in _cors_origins.split(",") if o.strip()]
else:
    _origins_list = ["http://localhost:5173", "http://127.0.0.1:5173"]

CORS(
    app,
    supports_credentials=True,
    origins=_origins_list,
    allow_headers=["Content-Type"],
    methods=["GET", "POST", "OPTIONS"],
    max_age=86400,
)
```

Parâmetro	Antes	Depois
<code>allow_headers</code>	<code>"*"</code>	<code>["Content-Type"]</code> — único header enviado pelo frontend
<code>methods</code>	implícito (todos)	<code>["GET", "POST", "OPTIONS"]</code> — nenhuma rota usa PUT/DELETE/PATCH
<code>max_age</code>	ausente (~5 min)	<code>86400</code> (24h) — reduz preflight OPTIONS repetidos
fallback dev	<code>origins="*"</code> implícito	<code>["localhost:5173", "127.0.0.1:5173"]</code> — explícito

Como verificar

DevTools → Network → pedido autenticado → cabeçalho `Access-Control-Allow-Origin` deve mostrar a origem do frontend (não `*`).

Teste negativo: `curl -H "Origin: https://evil.com" https://<render-url>/health` — ausência de `Access-Control-Allow-Origin` na resposta.

SEC-02 — HTTPS Obrigatório em Produção (12/05/2026)

O que é

Garantir que todos os pedidos à API são servidos em HTTPS e que o browser é instruído a recusar HTTP futuro (HSTS).

Porquê importa

Sem HTTPS, cookies de sessão podem ser interceptados em redes não seguras. HSTS garante que o browser nunca tente HTTP mesmo que o utilizador o escreva manualmente.

Implementação

- Render força HTTPS no reverse proxy — não existe porta HTTP exposta 
- `CHICHORRO_SESSION_SECURE=1` definida no Render → `SESSION_COOKIE_SECURE=True` (AUTH-06) 
- Header HSTS adicionado via `@app.after_request` em `Flask.py`:

```
if _session_secure:
    response.headers["Strict-Transport-Security"] = "max-age=31536000; includeSubDomains"
```

HSTS só é enviado em produção (`_session_secure=True`); não interfere em dev.

Como verificar

`curl -I https://<render-url>/health` → confirmar `Strict-Transport-Security: max-age=31536000; includeSubDomains` na resposta.

SEC-03 — Headers de Segurança, CSRF e CSP (12/05/2026)

O que é

Adição de headers HTTP de segurança via `@app.after_request`, análise da proteção CSRF existente e avaliação de CSP.

Porquê importa

- `X-Content-Type-Options`: impede MIME sniffing
- `X-Frame-Options`: impede clickjacking via `<i frame>`
- `Referrer-Policy`: limita informação de Referer em pedidos cross-origin

Implementação

```
@app.after_request
def _add_security_headers(response):
    response.headers["X-Content-Type-Options"] = "nosniff"
    response.headers["X-Frame-Options"] = "DENY"
    response.headers["Referrer-Policy"] = "strict-origin-when-cross-origin"
    if _session_secure:
        response.headers["Strict-Transport-Security"] = "max-age=31536000;
includeSubDomains"
    return response
```

CSRF: não foram adicionados tokens explícitos — proteção garantida por 4 camadas existentes: `Content-Type: application/json` obrigatório (`_guard_api_requests`) + CORS whitelist (SEC-01) + `SameSite=Lax` (AUTH-06) + `HttpOnly` (AUTH-06).

CSP: diferida para Cloudflare Pages — Flask só serve JSON; CSP aplica-se a documentos HTML.

Como verificar

`curl -I http://127.0.0.1:50/health` → confirmar `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: strict-origin-when-cross-origin` na resposta.